

## 2306. Naming a Company

Hard · Hash Table · String · Enumeration

---

### Approach — Suffix Grouping + Pair Counting

For each idea, consider its **suffix** (everything after the first letter). A  $26 \times 26$  matrix  $v[r][c]$  is built, where  $v[r][c]$  counts how many ideas starting with letter  $r$  can have their first letter swapped to  $c$  without conflicting with an existing idea in the list.

For every valid swap pair  $(i, j)$ , the number of valid company names is  $v[i][j] \times v[j][i]$ , because each suffix free to move from  $i \rightarrow j$  can be paired with every suffix free to move from  $j \rightarrow i$ . Summing all such products gives the final answer.

**Key Insight:** Two names  $(A, B)$  form a valid company only when both suffixes are mutually "free" — the suffix of  $A$  must not already exist starting with  $B$ 's first letter, and vice versa. This mutual freedom is exactly what  $v[i][j] \times v[j][i]$  captures.

### Step-by-step Breakdown

#### Step 1 — Build the hashmap

Insert all ideas into a `HashMap` for  $O(1)$  existence checks. This lets us instantly verify whether a modified word already exists in the idea list.

#### Step 2 — Populate the swap matrix

For each idea with original first letter  $r$ , try replacing it with every other letter  $c$  (skipping when  $r == c$ ). If the resulting word is *not* in the hashmap, increment  $v[r][c]$ .

#### Step 3 — Count valid pairs

Iterate all  $26 \times 26$  pairs  $(i, j)$  and add  $v[i][j] \times v[j][i]$  to the answer. This counts pairs where idea  $A$  (starting with letter  $i$ ) swaps to  $j$ , and idea  $B$  swaps back to  $i$  — both new names must be absent from the original list.

### Solution Code

```

1  class Solution {
2  public:
3      long long distinctNames(vector<string>& ideas) {
4
5          unordered_map<string, bool> hashMap;
6          for(auto& idea: ideas) hashMap[idea] = true;
7
8          vector<vector<int>> v(30, vector<int>(30));
9          for(auto& idea: ideas)
10         {
11             int r = idea[0] - 'a';
12             for(int c = 0; c < 26; c++)
13             {
14                 if(r == c) continue;
15                 idea[0] = 'a' + c;
16                 if(!hashMap.count(idea)) v[r][c]++;
17             }
18         }
19         long long ans = 0;
20         for(int i = 0; i < 26; i++)
21             for(int j = 0; j < 26; j++)
22                 ans += v[i][j]*v[j][i];
23         return ans;
24     }
25 };

```

## Complexity Analysis

| TIME COMPLEXITY                                                                                                                               | SPACE COMPLEXITY                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b><math>O(n \cdot 26)</math></b>                                                                                                             | <b><math>O(n + 26^2)</math></b>                                                  |
| For each of $n$ ideas, we try 26 letter swaps $\rightarrow O(26n)$ . The final $26 \times 26$ double loop is $O(676)$ , effectively constant. | The hashmap stores $n$ entries. The swap matrix is a fixed $26 \times 26$ array. |